Solution: Serialize and Deserialize Binary Tree

Problem Statement

`Serialization` involves converting a binary tree into a string format such that it can be stored in the file or memory, preserving its structure and data.

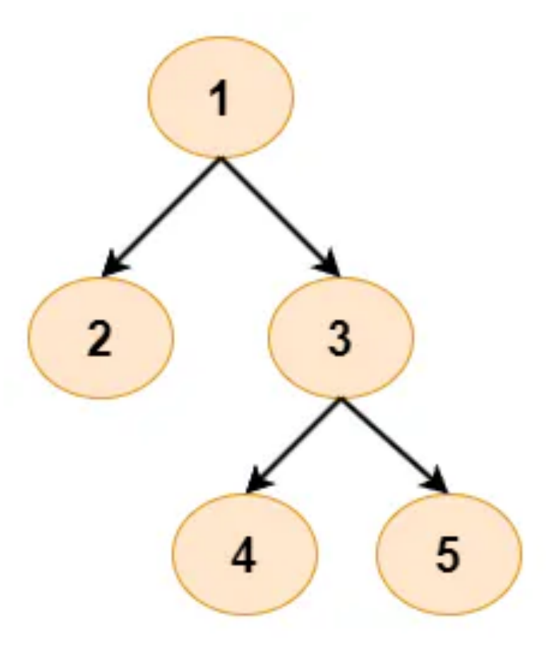
`Deserialization`, on the other hand, is about reconstructing the binary tree from this string representation.

Design an algorithm to `serialize` and `deserialize` the binary tree. There is no restriction on designing the algorithm for serialization and deserialization of the binary tree.

Examples

Example 1:

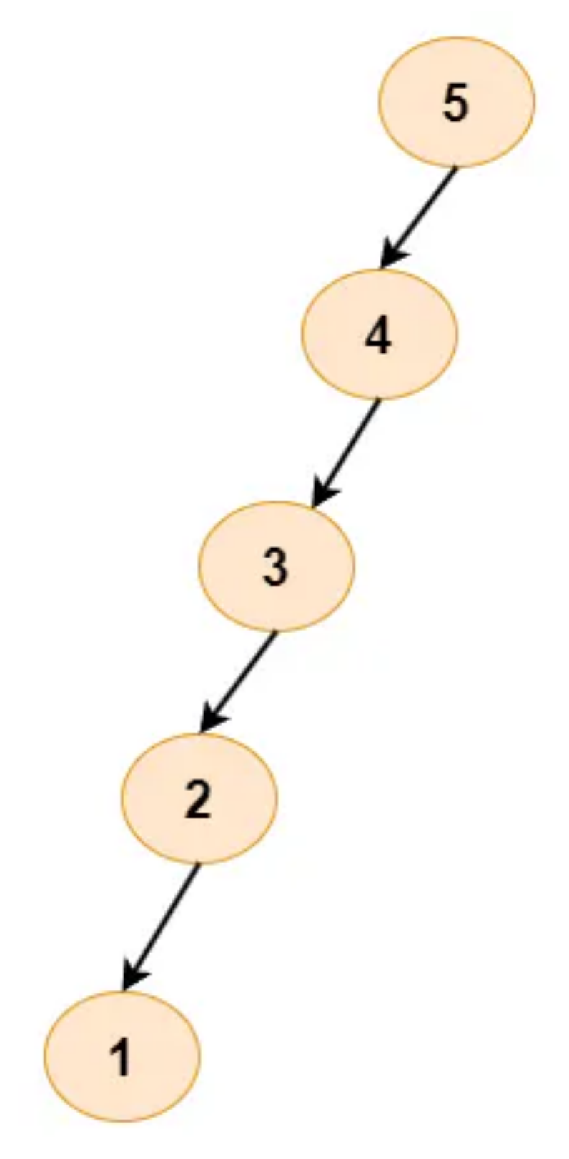
- **Input**: `[1,2,3,null,null,4,5]`



- **Expected Output**: `[1,2,3,null,null,4,5]`
- **Justification**: The tree starts with the root node 1, followed by its left child 2 and right child 3. Since 2 and 3 have no left children, they are followed by `null`. Nodes 4 and 5 are the left and right children of 3.

Example 2:

- **Input**: `[5,4,null,3,null,2,null,1]`



- **Expected Output**: `[5,4,null,3,null,2,null,1]`
- **Justification**: This represents a skewed tree where each node except the last one has a left child. The serialized format follows the tree structure exactly.

Example 3:

- **Input**: `[]`
- **Expected Output**: `[]`
- **Justification**: Since the tree is empty, both the serialized and deserialized outputs should be empty.

Solution

To solve this problem, we'll employ a breadth-first search (BFS) approach for serialization and a queue-based mechanism for deserialization. BFS is chosen for serialization because it naturally navigates through the tree levels, ensuring all nodes are covered and properly placed in the serialized string. For deserialization, using a queue enables us to reconstruct the tree level by level, maintaining the structure as in the serialized string. This method is effective as it straightforwardly maps the tree structure to a string format and vice versa, ensuring no data loss and maintaining the original tree structure.

Step-by-step Algorithm

Serialization:

1. **Initialize:** Create an empty string `result` and a queue `nodeQueue`.
2. **Check for Empty Tree:** If the root is `null`, return an empty string.
3. **Enqueue Root:** Enqueue the root node into `nodeQueue`.
4. **Process Queue:**
   - While `nodeQueue` is not empty:
     - **Dequeue Node:** Remove the front node from the queue.
     - **Append to String:**
       - If the node is `null`, append `"null"` to `result`.
       - If the node is not `null`, append the node's value.
     - **Enqueue Children:**
       - If the node is not `null`, enqueue its left and right children (including `null`).
5. **Return Result**: Return `result` as the serialized string.

Deserialization:

1. **Check for Empty String**: If the input string is empty, return `null` (representing an empty tree).
2. **Split String:** Split the input string into an array of values.
3. **Create Root:** Construct the root from the first value.
4. **Initialize Queue**: Create a queue `childQueue` and enqueue the root.
5. **Reconstruct Tree**:
   - Iterate through the array of values (starting from the second value):
     - **Assign Children:**
       - For each node in `childQueue`, assign the next two values as its left and right children.
       - Enqueue these children if they are not `null`.
     - **Remove Processed Node:** Dequeue the processed node.
6. **Return Root:** Return the root of the reconstructed tree.

Algorithm Walkthrough

Let's consider the Input: `[5,4,null,3,null,2,null,1]`

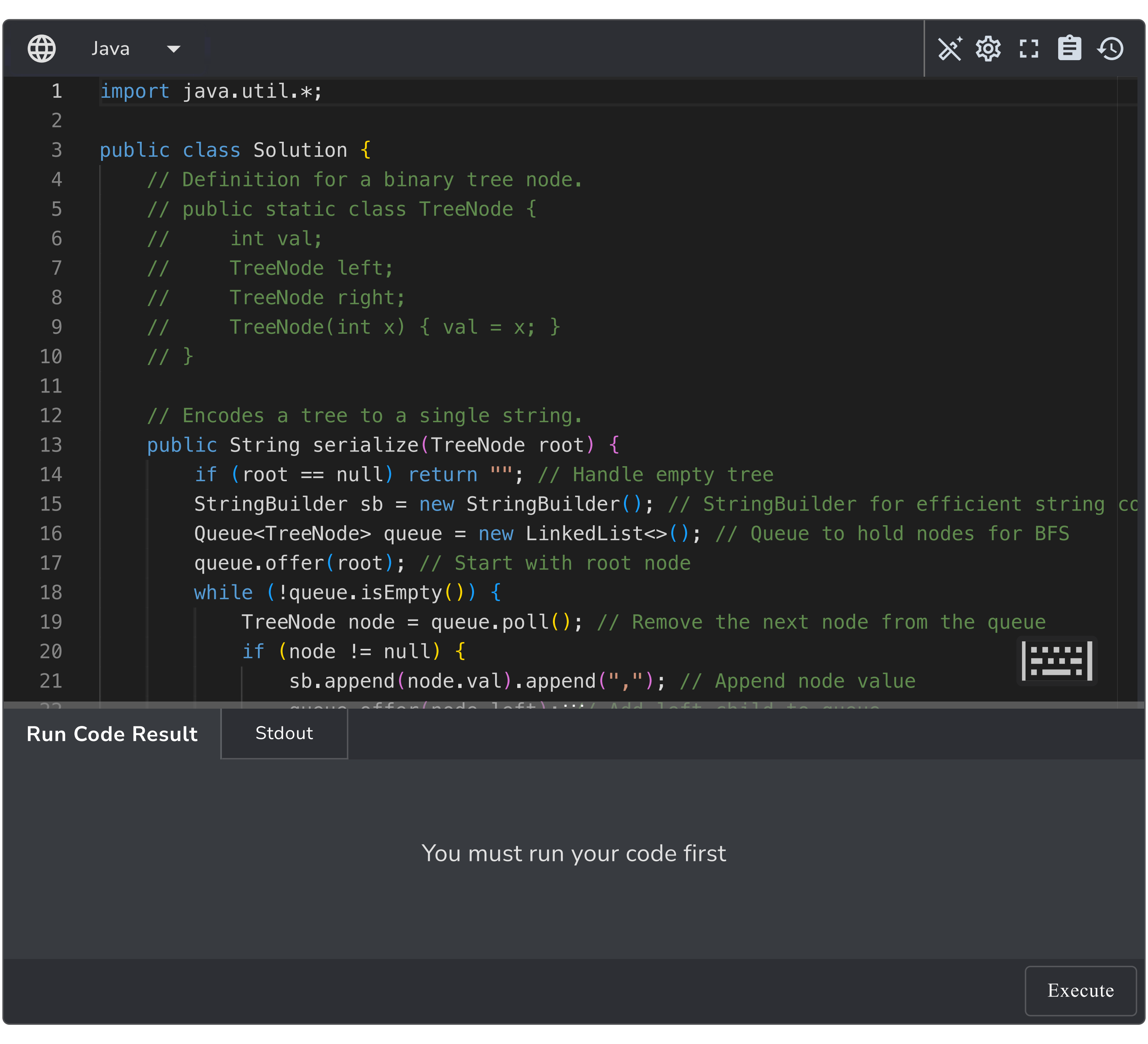
Serialization Walkthrough:

1. **Initialize**: `result = ""`, `nodeQueue = []`.
2. **Enqueue Root**: Enqueue 5.
3. **Process Queue:**
   - Dequeue 5 (front of queue), append "5" to `result`. Enqueue 4 (left child of 5) and `null` (right child of 5).
   - Dequeue 4, append "4" to `result`. Enqueue 3 (left child of 4) and `null` (right child of 4).
   - Dequeue `null`, append "null" to `result`. Since it's `null`, no children are enqueued.
   - Dequeue 3, append "3" to `result`. Enqueue 2 (left child of 3) and `null` (right child of 3).
   - Dequeue `null`, append "null" to `result`. No children to enqueue.
   - Dequeue 2, append "2" to `result`. Enqueue 1 (left child of 2) and `null` (right child of 2).
   - Dequeue `null`, append "null" to `result`. No children to enqueue.
   - Dequeue 1, append "1" to `result`. Both children are `null`, so nothing to enqueue.
   - Queue is now empty.
4. **Serialization Result:** `result = "5,4,null,3,null,2,null,1"`.

Deserialization Walkthrough:

1. **Split String**: `["5", "4", "null", "3", "null", "2", "null", "1"]`.
2. **Create Root**: Root = Node(5).
3. **Initialize Queue**: `childQueue = [Node(5)]`.
4. **Reconstruct Tree:**
   - Dequeue Node(5). Assign 4 as left child and `null` as right child. Enqueue Node(4).
   - Dequeue Node(4). Assign 3 as left child and `null` as right child. Enqueue Node(3).
   - Dequeue Node(3). Assign 2 as left child and `null` as right child. Enqueue Node(2).
   - Dequeue Node(2). Assign 1 as left child and `null` as right child. Enqueue Node(1).
   - Dequeue Node(1). No children to assign as both are `null`.
5. **Deserialization Result:** The tree structure is reconstructed to match the original input `[5,4,null,3,null,2,null,1]`.

Code



```java
import java.util.*;

public class Solution {
    // Definition for a binary tree node.
    // public static class TreeNode {
    //     int val;
    //     TreeNode left;
    //     TreeNode right;
    //     TreeNode(int x) { val = x; }
    // }

    // Encodes a tree to a single string.
    public String serialize(TreeNode root) {
        if (root == null) return ""; // Handle empty tree
        StringBuilder sb = new StringBuilder(); // StringBuilder for efficient string c
        Queue<TreeNode> queue = new LinkedList<>(); // Queue to hold nodes for BFS
        queue.offer(root); // Start with root node
        while (!queue.isEmpty()) {
            TreeNode node = queue.poll(); // Remove the next node from the queue
            if (node != null) {
                sb.append(node.val).append(","); // Append node value
```

You must run your code first

Complexity Analysis

Time Complexity

- **Serialization**: The time complexity for the given algorithm is `O(N)`, where N is the number of nodes in the tree. It is because we iterate through each node once using a queue.
- **Deserialization**: Each node value in the serialized string is processed exactly once. Thus, the time complexity is also `O(N)`, where N is the number of nodes represented in the serialized string.

Space Complexity

- **Serialization**: The space complexity of the algorithm is `O(N)` due to the queue holding a maximum of N/2 nodes at the widest level of the tree.
- **Deserialization**: The space required for the tree is `O(N)`, leading to a total space complexity of O(N).